MusicMind Stage 2 Progress Report

EECS3311 Software Design · Status on November 1, 2026 · Due November 10, 2026

12 of 12	657	1 of 9	9 days
Stage 1 features working through the CLI	automated tests, all passing on main	planned GUI screens built	left until submission

Bottom line. The backend and CLI are merged into **main** and work on the real Spotify account. The design matches Stage 1 closely, and the differences that were forced by Spotify's limits are recorded. The project is not ready to submit: the GUI is required and only its first screen exists, and the videos, updated UML and Stage 2 report have not been done.

Your four questions, answered

Question	Short answer
1. Is it aligned with Stage 1?	**Yes, closely.** Every application class in the Stage 1 diagrams exists under the same name, and all five planned design patterns are in the code. Six smaller gaps against Stage 1 remain and are listed in Section 1.
2. How is every feature working, and are the tests satisfying?	**Satisfying on the CLI, with limits.** 3 features fully meet their Stage 1 purpose and 9 mostly meet it. None fails. The weak spots are search relevance, thin genre data, and paths never tried on real Spotify.
3. What are the challenges ahead?	**Mainly the GUI and the submission package.** A full GUI, videos for every feature, the report and UML must all fit in nine days. Section 3 ranks them.
4. How much backend is done, and what is the front-end plan?	**About 95% of the backend, by my estimate.** The GUI will be one tabbed JavaFX window over the existing AppController, with wireframes in Section 4.

Overall progress (my estimates)



The percentages are judgment calls, not measurements. Measured facts in this report come from the merged code at commit 1c1b53d, the test results, and the project's test notes.

1. Alignment with Stage 1

The project was built against the Stage 1 report and UML, with each conflict in Stage 1 settled in a written ruling (R1 to R20). The table shows where it stands by design area.

Design area	Status	Evidence
12 features (F1 to F12)	**Aligned**	All implemented and reachable from the CLI. None was dropped or replaced.
Classes and names	**Aligned**	Every application class in the two Stage 1 class diagrams exists with its Stage 1 name. MockLLMProvider stays test-only by design. The GUI view classes do not exist yet.
Design patterns	**Aligned**	Adapter, Strategy, Command, State and Facade are in the code, plus MVC, Repository and an added Decorator (retry, genres).
Agent workflow (SD01 to SD04)	**Aligned**	Create runs through the Stage 1 states: planning, retrieving, selecting, validating, executing, then done or failed.
AI role	**Aligned**	The LLM interprets and words things. It never picks tracks. Selection, validation and storage are deterministic.
Spotify compliance	**Aligned**	Only ids and MusicMind's own derived data are stored, not titles or artist names. Spotify is credited in the output.
Stack	**Aligned**	Java 17, JavaFX, PostgreSQL and one LLM (OpenAI gpt-4o by default) behind LLMProvider.
GUI and CLI share one core	**Partly**	AppController is shared and holds no business logic, but only the Profile screen uses it from the GUI so far.

Changes from Stage 1, with the reason for each

Stage 2 allows changes when they are explained. These are the ones that affect what the app does.

Stage 1 design	What was built	Why
Genres come from Spotify artist data	Genres come from MusicBrainz through a Decorator	Spotify sends no genres to this app.
Search with Spotify's genre: filter	Genre sent as a plain keyword; mood words mapped to genres	Live, the filter returned 0 or 1 tracks where keywords returned 20.
Familiar meant tracks already in the library	Familiar means a known artist	Live, a create run had no new artists until this changed. The Stage 1 example now gives 8 of 27 new (30%).
Undo through an in-memory history	A saved snapshot before each edit, each used once by undo	Every CLI run is a new process, so memory alone cannot undo.
Validate against the original target	The target follows deliberate length edits, undo and restore	After "15 minutes shorter", validate wrongly reported a failure.
Main classes only	Small supporting classes added, such as TrackList and PlaylistTargets	They keep Stage 1 classes focused. No Stage 1 name changed.

Gaps still open against Stage 1

I found these while checking the code for this report. None stops a feature from working, but each is a difference that must be fixed or explained in the Stage 2 report.

Feature	Stage 1 says	Today	Recommendation
F3	Results include tracks and artists, each with "add to candidates"	Tracks only; no candidate list	Fix: add candidates in the GUI and feed them to Create. Document that results are tracks only.
F2	Differences across the three ranges	Compares newest with oldest; medium is shown but not compared	Fix if time allows, otherwise explain.
F7	Checks blocked artists and tracks	Blocks artists only	Fix: small addition to the rule and the preferences.
F8	Stores disliked artists	Takes Spotify artist ids, not names	Fix in the GUI with a name search.
F12	Opening a session shows its plan and result	Plan is stored, but history prints one line per session	Fix: show it in the History screen and the CLI.
F6	"Opening a playlist" to edit it	No call lists the user's playlists; the id must be pasted	Fix: add an owned-playlist list for the GUI.

2. Feature status and test results

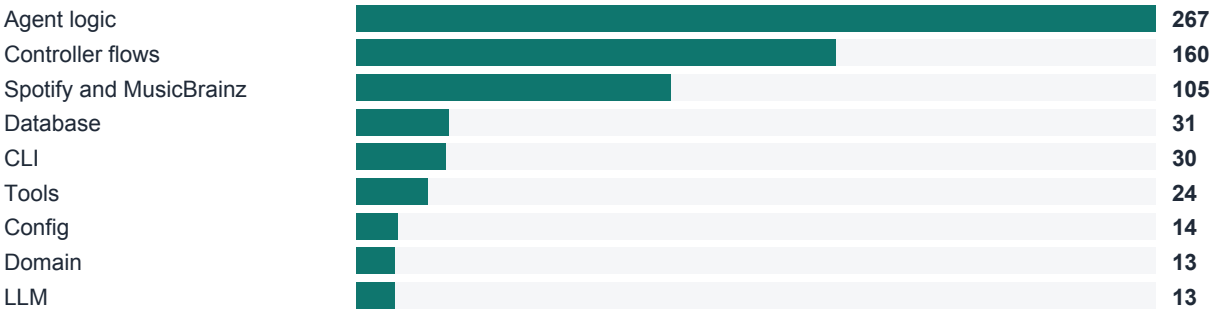
Each verdict compares today's CLI behaviour with the purpose stated in Stage 1. The GUI side of every feature except F1 is still missing, and is covered in Section 4.

Feature	Stage 1 purpose	How it works today (real account unless noted)	Verdict
F1 Taste Profile	Turn listening data into a genre and artist picture	Works, with an AI summary and a labelled fallback. MusicBrainz gave genres for 13 of 20 top artists; many Persian artists have none.	**Mostly met**
F2 Taste Change	Show how taste shifted over time	Works and says so when genre coverage is partial, rather than claiming a change. Medium range is not compared.	**Mostly met**
F3 Search	Turn free text into ranked results	Works, including a labelled keyword fallback. Spotify matches title words, so results can drift from a mood request.	**Mostly met**
F4 Discovery	Recommend new music with reasons	Works: excludes known tracks and gives a score and reason. Relevance is limited by the same title-word search.	**Mostly met**
F5 Create Playlist	Turn a goal into a real, valid playlist	Stage 1's own example gave a real playlist of 27 tracks, 8 by new artists (30%), passing validation.	**Met**
F6 Modify and Undo	Reversible edits from plain language	Edits work, and undo was checked live today with exact order restored. LLM cuts vary: "15 minutes shorter" removed from 1 to 5 tracks across runs.	**Mostly met**
F7 Validation	Report every rule violation	PASS live on created playlists. Duration, explicit, duplicates and length rules work. Blocked individual tracks are missing.	**Mostly met**
F8 Preferences	Keep standing preferences	Saves and applies them, and rejects bad values. Blocked artists need ids instead of names.	**Mostly met**
F9 Feedback	Learn from accept and reject	Saves verdicts, updates a repeat instead of duplicating, and changes later scores.	**Met**
F10 Explanation	Explain choices from real facts only	Words the recorded decisions and rejects text not supported by them, falling back to a labelled computed sentence.	**Met**

Feature	Stage 1 purpose	How it works today (real account unless noted)	Verdict
F11 Negotiation	Offer relaxations, never break rules silently	The real prompt appeared and cancelling changed nothing. Choosing an option has only run in automated tests.	**Mostly met**
F12 History, Restore	Undo across sessions	Restore and undoing it were checked live today, matching exactly. Recreating a deleted playlist has only run in automated tests.	**Mostly met**

Automated test results

657 tests pass, none fail and none are skipped, on a forced full run of **main**. They use an embedded PostgreSQL database and a local stand-in for Spotify's web service, so they need no keys or network.



Is that a satisfying result?

- **For correctness, yes.** The flagship goal from Stage 1 works end to end on real data, and safety rules hold: a failed run removes a half-made playlist, or gives you its id if it cannot, and undo restores exact order.
- **For music quality, partly.** Automated tests prove the logic, not whether a listener likes the songs. Spotify's limits on this app make relevance the weakest part of the product.
- **For coverage, not fully.** Passing tests use stand-ins, so real Spotify outages, a playlist with local files, recreating a deleted playlist and choosing an F11 option remain unproven live.

Live testing was worth the effort: it found problems that automated tests could not.

Found by	Problem	Result
Live run	Spotify returned no genres	MusicBrainz added
Live run	Create picked no new artists	Now 30% new on the Stage 1 example
Live run	Saved-track cap hid songs (library of 1,165)	Cap raised to 5,000
Code review	Repeated undo re-applied an edit	Fixed, confirmed live
Code review	Local files could make a move hit the wrong track	Fixed, tested over HTTP
You	Spotify key committed to .env.example	Key rotated and history rewritten; the old commit may still open by direct link

3. Challenges ahead

Ranked by risk to the November 10 submission.

Challenge	Why it matters	Plan
Building the full GUI	Stage 2 requires it, and mockups do not count. Eight of the nine screens and dialogs are still missing.	Build in the order in Section 4, with the main flows first. Each screen only calls AppController, so little new logic is needed.

Challenge	Why it matters	Plan
Keeping the window responsive	Create and Discover can take a while, and the first genre lookup is limited to about one artist per second.	Run every call on a background worker with visible progress. Never block the screen thread.
F11 inside the GUI	The agent pauses on a worker thread while the user chooses in a dialog on the screen thread.	Hand the choice between threads, and test choose, cancel and close-window.
Videos for every feature	Every feature needs a demo with a direct timestamp link, across GUI and CLI. Only you can record these.	Record after the GUI is stable. Keep .env and tokens off screen.
Report, UML and stale docs	The report needs seven sections. STAGE2_PLAN.md still contains old SQLite and Gemini text, and the README needs full setup steps.	Tasks 14 to 16. Use this report's gaps table so that every difference is explained.
Closing the six Stage 1 gaps	Unexplained differences are not accepted.	Fix them with the GUI work, or document them as limits.
Unproven live paths	Failure paths could still behave differently on real Spotify.	Try recreate and an F11 choice live on test playlists, with your approval.
Music quality in demos	LLM results vary between runs, and search leans on title words.	Rehearse goals known to work, and state the limit honestly in the report.
Running on another computer	A marker needs PostgreSQL, their own keys and Spotify Premium; Spotify allows 5 users on this app. Windows and non-Mac JavaFX are unverified.	Clear README steps, plus the videos as evidence. Test a clean install if possible.
Commit requirement	Period 1 (Oct 20 to 26) had 0 code commits, so the single 5-point deduction already applies and cannot be undone. Period 2 has 95.	Keep 5 real code commits for Nov 3 to 9 anyway. The GUI and fixes will provide them.
Items only you can do	Real AI token and money costs, making the repository public, deleting test playlists.	Do these before final checks on Nov 9.

Timeline

When	Work
Mon, Nov 2	Start the GUI on a new stage2-gui branch. The plan puts all of Tasks 13 to 17 on this day, which is ambitious, so I expect some to move into the buffer.
Nov 3 to 6	Finish the GUI, close the six gaps, and test the GUI on your account.
Nov 6 to 8	Docs, UML, Stage 2 report, and your videos.
Nov 9	Final review, secret scan, clean-build check, and links added to the report.
Nov 10	Submit the repository URL.

4. Backend completion and front-end plan

Backend

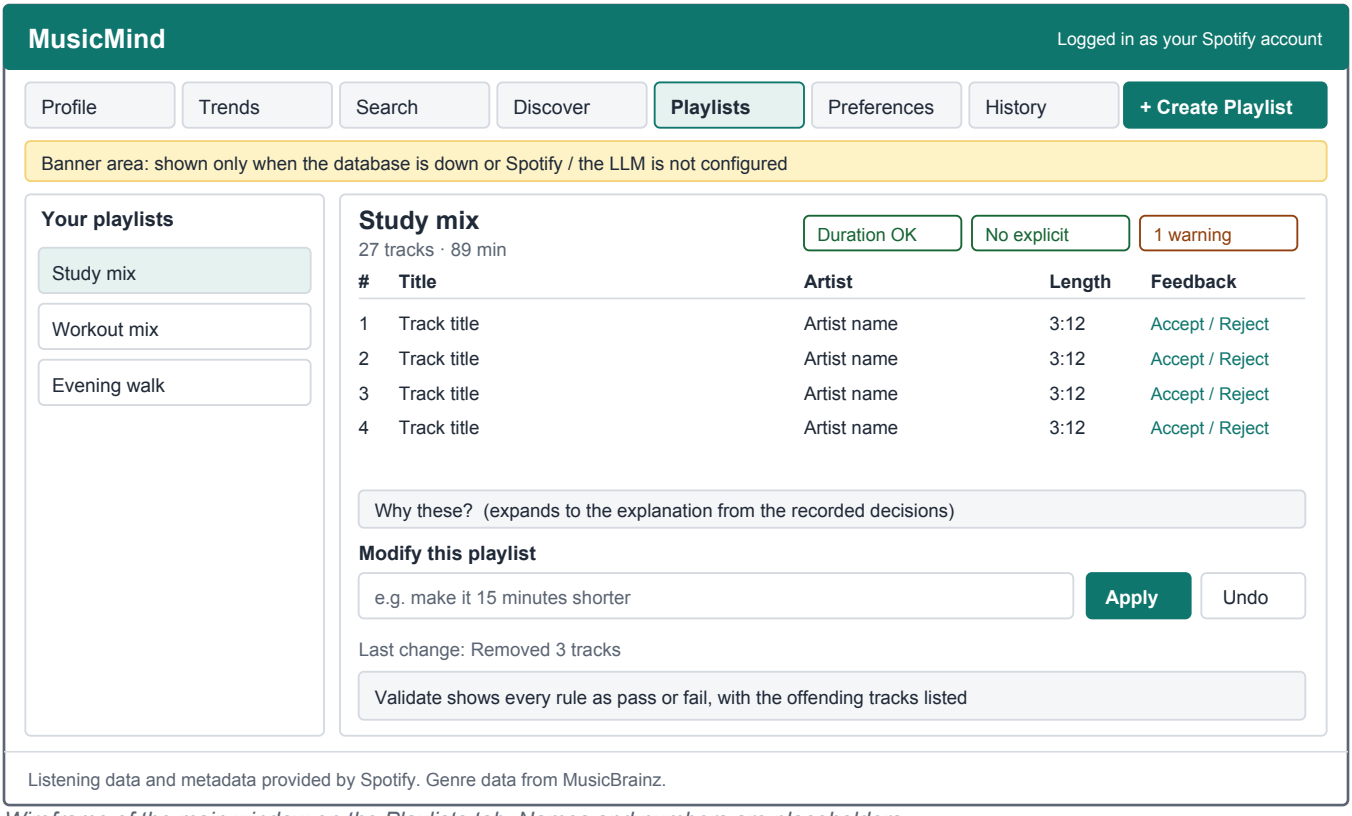
By my estimate the backend is **about 95% complete**. All 12 features run on it, and the remaining 5% is the small additions listed below. It is about 11,400 lines of application code in 110 files, with about 10,700 lines of test code.

Part	Status	Lines	What it covers
Spotify and MusicBrainz	**Done**	1,604	PKCE login, Feb 2026 endpoints, retries, genres, caching
Agent and recommendation	**Done**	3,067	Planning, checks, recommender, three strategies, explanations
Session states	**Done**	632	The full create lifecycle with negotiation and re-login
Playlist commands	**Done**	865	Add, remove, replace, reorder, create, restore, undo, recovery
Database	**Done**	914	PostgreSQL, repositories, snapshots; safe defaults when it is down
Controller and facade	**Done**	1,812	One entry point for every feature, shared with the GUI
CLI	**Done**	828	All 16 commands, interactive mode, exit statuses
Domain, tools, LLM, config	**Done**	1,651	Data model, agent tools, OpenAI provider, settings
GUI	**Started**	141	One screen for F1

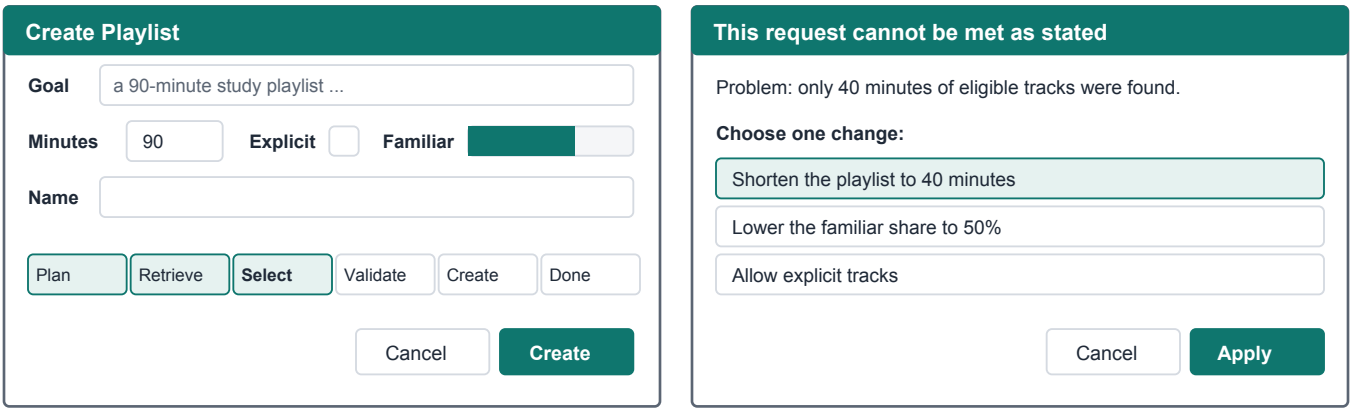
Backend work still needed for the GUI: a list of owned playlists, a candidate set for F3, blocked tracks for F7, an artist-name lookup for F8, session detail for F12, and a read of a playlist's tracks with display titles.

Front-end design

Stage 1 describes tabs, panels and dialogs built as views over a shared core, so the GUI will be **one JavaFX window with a tab bar**, and it will reuse the commands and wording of the CLI. Views hold no business logic.



Wireframe of the main window on the Playlists tab. Names and numbers are placeholders.



Wireframes of the Create Playlist dialog with live steps (left) and the F11 negotiation dialog (right).

Screens

Screen	Features	Main controls
ProfileView	F1	Time range, genre chart, top artists and tracks, summary, Refresh profile
TrendsView	F2	Three side-by-side range columns, narrative, coverage note
SearchView	F3	Search bar, ranked results, Add to candidates
DiscoveryView	F4, F9, F10	Novelty slider, recommendations with reasons, Accept and Reject
CreatePlaylistDialog	F5	Goal, minutes, explicit toggle, familiar ratio, name, live step indicator
NegotiationDialog	F11	Conflict, relaxation options, Apply, Cancel; used by Create and Modify
PlaylistView	F6, F7, F9, F10	Track table, validation badges, Modify box, Undo, Why these?, Accept and Reject
PreferencesView	F8	Blocked artists, styles, explicit policy, familiar leaning
HistoryView	F12	Session list, plan and result, saved versions, Restore, Recreate

How it will work

- **Structure:** one class per view in the app package (MVC), each calling only AppController, so the CLI and GUI cannot drift apart.
- **Responsiveness:** a shared background-task helper handles the spinner, disables the button while running, and shows errors in plain words.
- **Live progress:** the Create steps follow the same session states the CLI prints, so the State pattern is visible to the user.
- **Failures:** a banner covers a database outage or missing configuration, and login opens the browser only when needed.
- **Look:** a clean light theme in one stylesheet, with teal as the accent so that it does not imitate Spotify's branding. Status always uses words as well as colour.
- **Build order:** shared shell and helper; Profile, Trends and Preferences; Search and Discover; Create and Negotiation; Playlists; History.
- **Checks:** unit tests for display helpers, a launch test with no crash, then a full pass through every screen on your account before recording.